

RateBuzz - Help Guide

Rate Intelligence Platform | Frontend Reference & Integration Roadmap

1. Project Structure

RateBuzz is a frontend-only React application (React 18 + react-router-dom) that renders directly in-browser via a service worker performing an in-request Babel transform - there is no npm install and no bundler build step.

- src/pages/{properties,rooms,ratePlans}/ - List page, Form, Detail/Profile view per entity.
- src/pages/settings/ - General, Defaults, Appearance, Integrations.
- src/components/layout/ - Sidebar, Topbar, PortalLayout, AuthGuard.
- src/components/ui/ - shared design-system components (Table, Modal, Badge, FeatureChipGrid, MasterDataManager, etc).
- src/context/ - AuthContext (mock login/roles), DataContext (CRUD + scoping + master data), PropertyContext, ToastContext, AppearanceContext.
- src/lib/ - permissions, formatting, pricing-period, and export helpers.
- src/mocks/ - seed data for properties, rooms, rate plans, and master tables (room types, amenities, room templates).
- src/hooks/ - usePermissions, useSelection, useUnsavedChanges, usePersistedState, useCursorGlow.
- src/styles/ - plain CSS design system (tokens.css for design tokens, remainder split by concern).

2. Installation

No package installation is required. The project uses import maps (declared in index.html) to load React, ReactDOM, and react-router-dom from a CDN at runtime, and a service worker (sw.js) transforms JSX on the fly with Babel.

- Clone or copy the ratebuzz/ folder.
- Windows PowerShell prerequisite: if serve.ps1 reports "not digitally signed", run:
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass (session-only, not machine-wide).

3. Running the Frontend

Two supported ways to start the static file server on http://localhost:5173/:

- Double-click start.bat (applies the execution-policy bypass automatically), or
- Run manually: powershell -ExecutionPolicy Bypass -File serve.ps1

Then sign in from the login screen using one of the two demo accounts described in Section 7 (User Roles), or use the Quick Demo Access buttons.

4. Folder Organization

The codebase is organized by feature/domain rather than by file type, so each entity (Properties, Rooms, Rate Plans) is self-contained under src/pages/<entity>/ with its list page, form, and profile page together. Shared, reusable pieces (buttons, tables, modals, filter panels, master-data managers) live under src/components/ui/ and are imported wherever needed. This mirrors how an MVC project typically separates Controllers/Views by feature area, which keeps the eventual migration mechanical rather than a redesign.

5. Future .NET Integration

Target backend: .NET Core 8.0 (C#), Framework: ASP.NET MVC. The frontend has been deliberately kept framework-agnostic at the markup level:

- Views remain uncompiled JSX served directly by a service worker - no bundler-specific output, no hashed asset names, no build artifacts to reconcile against Razor Views. Each .jsx page can be ported into an MVC View (or a View Component) largely as-is, keeping JSX-shaped markup as a guide for the eventual .cshtml structure.
- All data access is already centralized behind `src/context/DataContext.jsx` and a small set of hooks - no component calls `fetch/localStorage` directly for domain data. Swapping `DataContext`'s in-memory reducer for real HTTP calls to MVC controllers (Properties, Rooms, RatePlans, Masters) is a localized change.
- `src/lib/permissions.js` is the single seam for role/claims logic, designed to be replaced by ASP.NET Identity / JWT claims without touching any consuming component.

6. Future SQL Server Integration

Target database: NEW DATABASE: RATEBUZZ, on SQL Server 2022.

Every mutable collection in `DataContext` (properties, rooms, ratePlans, and the master-data slices: roomTypes, amenities, roomTemplates) maps directly onto a candidate SQL Server table of the same shape. The master-data CRUD (add/update/delete) added to the frontend already models the exact contract a `/api/masters/{kind}` MVC controller backed by SQL Server master tables would expose (id, name, and - for templates - a JSON/normalized values payload), so no frontend redesign is needed when the backend lands - only `DataContext`'s dispatch calls change from local reducer actions to HTTP calls.

7. Mock Data Architecture

`src/mocks/` holds seed arrays for every entity, structured close to how backend rows would look (id, name, and RBAC-relevant fields like `ownerId/propertyId`). `src/context/DataContext.jsx` wraps these in a `useReducer` store with a `stamp()` helper that timestamps and attributes every write (`lastModifiedBy/lastModifiedAt`) exactly as an API layer would. RBAC scoping (which properties/rooms/rate plans a signed-in user may see) happens once in `DataContext` via `scopedProperties/scopedRooms/scopedRatePlans`, so every page reads pre-scoped data instead of re-implementing an `ownerId` check.

8. User Roles

Two demo accounts are available from the login page or the Topbar account switcher (mock, demo-only - stands in for ASP.NET Identity/JWT roles until the backend exists):

- Super Admin (`admin@ratebuzz.com / Admin@123`) - full access to all properties, rooms, rate plans, and integrations.
- Property Owner (`owner@ratebuzz.com / Owner@123`) - scoped to their own properties only; cannot see Integrations; Property ID is locked after creation; can archive but not permanently delete Rooms/Rate Plans.

Capability flags are exposed via `usePermissions()`, derived from role in `src/lib/permissions.js`.

9. Module Overview

- Properties - list, create/edit form, and a tabbed profile (Overview, Contact, Address,

Operational, Rooms, Rate Plans, Notes, Settings). Archive/Restore + role-gated permanent delete. CSV export, Import Wizard.

- Rooms - property-scoped list with a fixed left filter panel (Properties, Room Type, Occupancy). Creating a room while multiple properties are selected clones an independent room record into each selected property. Room Types, Amenities, and Room Templates are managed as reusable master records via inline Add/Edit/Delete, architected as future SQL Server master tables.

- Rate Plans - property/room-scoped list with a nested Property to Room filter tree and a Meal Plan filter. The Cancellation Policy step supports an optional Partial Refund (Yes/No) with Refund Percentage and a Refund Until window (Hours/Days before check-in), mirroring the existing Tax Inclusive / Tax Percent interaction pattern. Pricing Periods are managed on the Rate Plan Profile page.

- Settings - General, Defaults (per-entity), Appearance (density/accent, persisted), and Integrations (role-gated).

Appendix - Key Files

- src/context/DataContext.jsx - single source of truth for all domain + master data, CRUD actions, and RBAC scoping.
- src/lib/permissions.js - role to capability-flag mapping (the future Identity/claims seam).
- src/mocks/masterData.js - seed data for the Room Types / Amenities / Room Templates master tables.
- src/components/ui/MasterDataManager.jsx - generic Add/Edit/Delete UI for simple {id, name} master tables.